

Museum Heist: A Grid-Based Stealth Game

Integrating Greedy Search, Backtracking, Linked Lists, and Binary Search Trees

Nelson de Jesus Navarro De La Rosa, Henry Samuel Garrido Medina, Josue Urrego Lopez
Computer Science I, Systems Engineering Program
Universidad Distrital Francisco Jose de Caldas, Bogota, Colombia

Abstract—Museum Heist is a Computer Science I final project implemented as a fully playable grid-based stealth game. The system integrates Python, Pygame, C++17, and JSON-file inter-process communication to demonstrate course concepts through interactive game mechanics rather than isolated code examples. The player navigates a museum floor plan, collects valuable items, evades camera and guard vision zones, and reaches an exit before losing by alarm, movement limit, or time limit. A C++ rule engine validates every action and owns the authoritative game state; Python renders the graphical interface, executes a greedy item recommendation, and computes a backtracking escape route. Custom C++ data structures—a singly linked list for movement history and a binary search tree (BST) ordered by item value—satisfy the course data-structure requirements. The final game supports three difficulty levels, dynamic vision zones, item collection with scoring, win/loss conditions, final rank calculation, a tutorial screen, restart, and main-menu navigation.

Index Terms—greedy algorithm, backtracking, linked list, binary search tree, Pygame, C++17, JSON inter-process communication, grid-based stealth game.

I. INTRODUCTION

Algorithms and data structures are most effectively understood when they drive visible, interactive behaviour rather than serving as abstract examples. Museum Heist was designed with this principle in mind: every required course concept—a greedy heuristic, a backtracking search, a linked list, and a BST—maps directly to a mechanic the player can see and feel during gameplay.

The game places the player in a museum grid. Guards occupy fixed positions and project vision cones in all four cardinal directions. Cameras project vision in a single fixed direction. Items of varying monetary value are scattered across the floor. The player must reach the exit cell while optionally collecting items, keeping an alarm counter below the per-difficulty maximum, staying within a movement budget, and completing the run before the clock expires.

The project is organised as a two-process system: a compiled C++ binary handles all rule logic and produces a canonical state file; a Python/Pygame front-end reads that file, runs the two Python algorithms, and renders everything to the screen. Communication between the two processes is entirely file-based through two JSON files, making the data flow observable and debuggable without a network stack or shared memory.

II. PROJECT REQUIREMENTS AND SCOPE

Table I maps each course requirement to the file and structure that satisfies it.

TABLE I
REQUIREMENT COVERAGE

Requirement	Implementation
Greedy algorithm	<code>greedy.py</code> : selects the single highest-priority uncollected item each turn based on proximity to guards and vision-zone safety.
Backtracking	<code>backtracking.py</code> : depth-first search from player to exit avoiding all blocked cells; chooses/explores/unchooses.
Linked list	<code>MovementList</code> in <code>linked_list.h/.cpp</code> : custom singly linked list; <code>push_back</code> in $O(1)$ via tail pointer.
BST	<code>ItemBST</code> in <code>tree.h/.cpp</code> : custom BST keyed by item value; inorder traversal produces the sorted item array in <code>state.json</code> .
JSON bridge	<code>input.json</code> carries action + preserved state to C++; <code>state.json</code> carries the full updated state back to Python.
Pygame UI	<code>main.py</code> : renders five screens, board, HUD, vision overlays, backtracking route, and greedy highlight.

III. SYSTEM ARCHITECTURE

A. Directory Layout

The repository is organised as follows:

```
museum-heist/  
  engine/  
    main.cpp           # C++ rule engine (1 106 lines)  
    linked_list.h/.cpp # custom linked list  
    tree.h/.cpp        # custom BST  
  game/  
    algorithms/  
      greedy.py        # greedy item selector  
      backtracking.py  # backtracking escape router  
    data/  
      input.json       # Python -> C++ channel  
      state.json       # C++ -> Python channel  
    ui/  
      main.py         # Pygame renderer + event loop  
      bridge.py       # subprocess + JSON I/O  
      requirements.txt # pygame only
```

B. Execution Cycle

Each player action triggers one full cycle (Fig. 1):

- 1) **Event capture.** Pygame detects a keyboard event (KEYDOWN) or a timer tick.
- 2) **Serialise.** `bridge.write_input()` writes `input.json` with the action, direction, difficulty, and all state fields that C++ must not lose (movement history, collected items, score, alarm, elapsed seconds).
- 3) **Engine run.** `bridge.run_engine()` invokes the compiled `museum_engine` binary via `subprocess.run()`.
- 4) **Deserialise.** `bridge.load_state()` reads `state.json` and returns a Python dict.
- 5) **Algorithm pass.** `choose_greedy_item(state)` and `find_escape_route(state)` are called; results feed the renderer.
- 6) **Render.** Pygame draws the updated board, vision overlays, route, greedy highlight, and HUD.

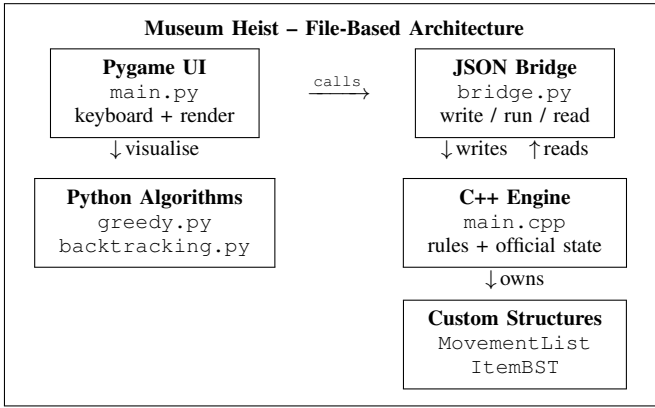


Fig. 1. File-based inter-process architecture.

C. Design Rationale

The file-based IPC was chosen for explainability: any state snapshot is human-readable JSON and can be inspected in a text editor. Python was kept as the UI and algorithm layer because Pygame provides the rendering primitives and Python’s dynamic typing shortens algorithm prototyping. C++ was used for the rule engine to fulfil the course requirement of implementing the linked list and BST in a compiled, memory-managed language.

IV. C++ RULE ENGINE

The engine is the single source of truth for game state. It is compiled with:

```
g++ -std=c++17 engine/main.cpp engine/linked_list.cpp \
    engine/tree.cpp -o engine/museum_engine
```

A. Data Structures Defined in `main.cpp`

The engine defines five plain structs (no STL containers used for game logic):

```
1 struct Position { int row; int col; };
2 struct ItemData { std::string id; int row,col,value; };
3 struct CameraData{ std::string id,direction;
4                     int row,col,range; };
5 struct GuardData { std::string id; int
6                     row,col,vision_range; };
7 struct MapData {
8     std::string difficulty;
9     int rows, cols, max_alarm,
10     max_movements, time_limit,
11     wall_count, camera_count,
12     guard_count, item_count;
13     Position start, exit;
14     Position walls [MAX_WALLS];
15     CameraData cameras[MAX_CAMERAS];
16     GuardData guards [MAX_GUARDS];
17     ItemData items [MAX_ITEMS];
18 };
```

Listing 1. Core structs in `engine/main.cpp`.

Fixed-size arrays (with compile-time constants `MAX_WALLS=16`, `MAX_CAMERAS=3`, etc.) are used instead of `std::vector` deliberately: no dynamic allocation happens inside the map structures, keeping the memory layout predictable and teaching array-based aggregation.

B. Hardcoded Maps and Difficulty Selection

Three maps are compiled into the binary. Table II summarises their parameters. The engine reads the difficulty field from `input.json` and calls the corresponding loader (`load_easy_map`, `load_normal_map`, `load_hard_map`).

TABLE II
DIFFICULTY PARAMETERS

Difficulty	Grid	Alarm	Moves	Time	Items
Easy	8 × 8	4	45	90 s	3
Normal	8 × 8	3	35	60 s	3
Hard	10 × 10	2	28	45 s	4

C. Manual JSON Parser

The engine reads `input.json` without an external library. Three helper functions provide the necessary primitives:

```
1 std::string get_string_value(const std::string& text,
2                             const std::string& key);
3 int get_int_value (const std::string& text,
4                   const std::string& key, int def);
5 bool get_bool_value (const std::string& text,
6                     const std::string& key, bool def);
```

Listing 2. Manual JSON field readers.

Each function locates the key with `std::string::find`, advances past the colon, then extracts the value up to the closing delimiter. While fragile against malformed JSON, this approach is self-contained and requires no third-party headers—an intentional pedagogical choice.

D. Movement Validation Pipeline

`apply_input()` is the central function of the engine. It reads every field from `input.json`, rebuilds the movement history into the `MovementList`, and—if the action is "move"—applies the following validation chain:

```

1 // 1. Reconstruct state from JSON
2 rebuild_movement_history(input, history, map);
3 read_collected_items(input, collected_items,
4   collected_count);
5 score = get_int_value(input, "score", 0);
6 alarm = get_int_value(input, "alarm", 0);
7 elapsed = get_int_value(input, "elapsed_seconds", 0);
8 // 2. Time-limit check (applies to every action type)
9 if (!escaped && !game_over &&
10   elapsed_seconds >= map.time_limit) {
11   game_over = true;
12   game_over_reason = "Time expired";
13   return;
14 }
15 // 3. Compute next cell
16 if (direction=="up") next_row--;
17 else if (direction=="down") next_row++;
18 else if (direction=="left") next_col--;
19 else if (direction=="right") next_col++;
20
21 // 4. Out-of-bounds: silently ignored
22 if (!next_cell_is_inside) return;
23
24 // 5. Wall: alarm++ and check limit
25 if (is_wall_cell(map, next_row, next_col)) {
26   alarm++;
27   check_alarm_limit(alarm, map.max_alarm,
28     game_over, game_over_reason);
29   return;
30 }
31 // 6. Camera / guard / vision: instant game over
32 if (is_camera_cell(...) || is_guard_cell(...) ||
33   is_vision_cell(...)) {
34   game_over = true;
35   game_over_reason = "Caught in vision zone";
36   return;
37 }
38 // 7. Valid move: update position and log
39 player_row = next_row; player_col = next_col;
40 history.push_back(player_row, player_col, step);
41
42 // 8. Adjacency alarm (next to guard without vision)
43 if (is_next_to_guard(map, player_row, player_col))
44   alarm++;
45
46 // 9. Item collection
47 collect_item_if_needed(...);
48
49 // 10. Escape / movement-limit checks
50 if (player_row==map.exit.row &&
51   player_col==map.exit.col) escaped = true;
52 check_alarm_limit(alarm, map.max_alarm, ...);
53 if (!escaped && !game_over &&
54   history.get_size() >= map.max_movements) {
55   game_over = true;
56   game_over_reason = "Movement limit reached.";
57 }

```

Listing 3. Movement validation chain in apply_input().

E. Vision Zone Computation

Camera vision propagates in one direction until it hits a wall or the grid boundary. Guard vision propagates in all four cardinal directions up to vision_range. Both use the same helper:

```

1 void write_camera_vision(const MapData& map,
2   std::ostream& out, bool& first){
3   for (int ci=0; ci<map.camera_count; ci++){
4     CameraData cam = map.cameras[ci];
5     int dr,dc;
6     get_direction_change(cam.direction, dr, dc);
7     int vr=cam.row, vc=cam.col;
8     for (int s=1; s<=cam.range; s++){
9       vr+=dr; vc+=dc;
10      if (out_of_bounds(vr,vc,map)) break;
11      if (is_wall_cell(map,vr,vc)) break;
12      write_vision_cell(out,first,cam.id,"camera",vr,vc);

```

```

13 }
14 }
15 }

```

Listing 4. Camera vision projection loop.

The resulting vision_zones array is written to state.json and consumed by both the Pygame renderer (for the orange overlay) and the Python algorithms (to classify cells as safe or dangerous).

F. Item Collection and Alarm Propagation

```

1 void collect_item_if_needed(
2   const MapData& map, int pr, int pc,
3   std::string collected[], int& count,
4   int& score, int& alarm){
5   for (int i=0; i<map.item_count; i++){
6     ItemData item = map.items[i];
7     if (pr==item.row && pc==item.col &&
8       !item_is_collected(collected, count, item.id)){
9       collected[count++] = item.id;
10      score += item.value;
11      if (item_is_near_guard(map, item))
12        alarm++; // near-guard penalty
13    }
14  }
15 }

```

Listing 5. collect_item_if_needed().

An item is considered *near a guard* when its Manhattan distance to any guard is ≤ 2 . Collecting such an item costs one alarm point, creating a risk/reward tradeoff for the player.

G. Rank System

After a successful escape, calculate_rank() evaluates three criteria:

```

1 std::string calculate_rank(const MapData& map,
2   bool escaped, bool game_over, int score,
3   int moves, int elapsed, int alarm){
4   if (game_over && !escaped) return "Caught";
5   if (!escaped) return "In Progress";
6   int total = total_item_value(map);
7   if (score >= total*7/10 &&
8     moves <= map.max_movements/2 &&
9     elapsed<= map.time_limit/2 &&
10    alarm <= 1) return "A";
11   if (score >= total*4/10 &&
12     moves <= map.max_movements &&
13     elapsed<= map.time_limit) return "B";
14   return "C";
15 }

```

Listing 6. Rank calculation.

H. State Output

write_state_file() serialises the entire game state to state.json using std::ofstream and manual string construction. Before writing, it populates the ItemBST with the uncollected items so that the inorder traversal produces the items_sorted_by_value field automatically. The MovementList::write_json_array() method serialises movement history directly into the output stream in $O(n)$.

V. JSON BRIDGE (BRIDGE.PY)

bridge.py encapsulates the entire inter-process contract in five functions.

```

1 PROJECT_ROOT = Path(__file__).resolve().parents[2]
2 ENGINE_PATH = PROJECT_ROOT / "engine" / "museum_engine"
3 INPUT_PATH = PROJECT_ROOT / "game" / "data" /
  "input.json"
4 STATE_PATH = PROJECT_ROOT / "game" / "data" /
  "state.json"
5
6 def compile_engine():
7     command = ["g++", "-std=c++17",
8               "engine/main.cpp", "engine/linked_list.cpp",
9               "engine/tree.cpp", "-o", "engine/museum_engine"]
10    subprocess.run(command, cwd=PROJECT_ROOT, check=True)
11
12 def run_engine():
13    subprocess.run([str(ENGINE_PATH)],
14                  cwd=PROJECT_ROOT, check=True)
15
16 def load_state():
17    with STATE_PATH.open("r", encoding="utf-8") as f:
18        return json.load(f)
19
20 def write_input(action, direction, state,
21                elapsed_seconds=None):
22    player = state["player"]
23    escaped = state.get("escaped", False)
24    game_over = state.get("game_over", False)
25    if action=="move" and (escaped or game_over):
26        action, direction = "none", ""
27    payload = {
28        "action": action, "direction": direction,
29        "difficulty": state.get("difficulty", "Normal"),
30        "player": {
31            "row": player["row"], "col": player["col"]},
32        "movement_step": len(state["movement_history"]),
33        "movement_history": state["movement_history"],
34        "collected_items":
35            state.get("collected_items", []),
36        "score": state.get("score", 0),
37        "escaped": escaped, "game_over": game_over,
38        "game_over_reason":
39            state.get("game_over_reason", ""),
40        "alarm": state.get("alarm", 0),
41        "elapsed_seconds": elapsed_seconds
42        if elapsed_seconds is not None
43        else state.get("elapsed_seconds", 0),
44    }
45    with INPUT_PATH.open("w", encoding="utf-8") as f:
46        json.dump(payload, f, indent=2)
47
48 def get_game_state(): # startup only
49    compile_engine()
50    run_engine()
51    return load_state()

```

Listing 7. bridge.py – complete implementation.

Key design decisions.

- write_input() guards against sending "move" when the game is already over, converting it to "none" so the engine skips processing without an error.
- compile_engine() is called only at startup via get_game_state(), not on every move, so compilation cost is paid once per session.
- All paths are resolved relative to PROJECT_ROOT using pathlib.Path, making the project relocatable.

Table III summarises the fields of both JSON files.

VI. CUSTOM C++ DATA STRUCTURES

Neither std::list nor std::map/std::set is used for the required structures.

TABLE III
JSON FILE SCHEMAS

input.json (Python → C++)

action	"move" "reset" "tick" "none"
direction	"up" "down" "left" "right" " "
difficulty	"Easy" "Normal" "Hard"
player	{row, col}
movement_step	integer – current step count
movement_history	array of {step, row, col} objects
collected_items	array of item id strings
score, alarm	integers
escaped, game_over	booleans
game_over_reason	string
elapsed_seconds	integer

state.json (C++ → Python)

grid	{rows, cols}
player, exit	{row, col}
walls	array of {row, col}
cameras	array of {id, row, col, direction, range}
guards	array of {id, row, col, vision_range}
vision_zones	array of {source, type, row, col}
items	uncollected items with value
items_sorted_by_value	BST inorder output
movement_history	full linked-list serialisation
status, rank	"Playing" "Escaped" "Caught" / A/B/C
score, alarm, elapsed_seconds	integers
max_alarm, max_movements, time_limit	integers

A. MovementList – Singly Linked List

1) Interface (linked_list.h):

```

1 struct MovementNode {
2     int row, col, step;
3     MovementNode* next;
4 };
5
6 class MovementList {
7     MovementNode* head;
8     MovementNode* tail;
9     int size;
10 public:
11     MovementList();
12     ~MovementList();
13     void push_back(int row, int col, int step); // O(1)
14     void clear(); // O(n)
15     void write_json_array(std::ostream& out) const; // O(n)
16     int get_size() const; // O(1)
17 };

```

Listing 8. MovementNode and MovementList declarations.

2) Implementation highlights (linked_list.cpp):

```

1 void MovementList::push_back(int row, int col, int step) {
2     MovementNode* node = new MovementNode;
3     node->row=row; node->col=col;
4     node->step=step; node->next=nullptr;
5     if (head==nullptr){ head=node; tail=node; }
6     else { tail->next=node; tail=node; }
7     size++;
8 }

```

Listing 9. push_back using tail pointer.

```

1 void MovementList::clear(){
2     MovementNode* cur = head;
3     while (cur != nullptr){
4         MovementNode* nxt = cur->next;
5         delete cur;
6         cur = nxt;
7     }
8     head=nullptr; tail=nullptr; size=0;
9 }

```

Listing 10. clear() – manual deallocation.

```

1 void MovementList::write_json_array(std::ostream&
2     out) const{
3     out << "[";
4     MovementNode* cur = head;
5     while (cur != nullptr){
6         out << "{\"step\":\"" << cur->step
7         << "\",\"row\":\"" << cur->row
8         << "\",\"col\":\"" << cur->col << "}";
9         if (cur->next) out << ",";
10        cur = cur->next;
11    }
12    out << "]";

```

Listing 11. write_json_array() – O(n) serialisation.

3) Complexity			summary:
Operation	Time	Notes	
push_back	O(1)	tail pointer avoids traversal	
clear	O(n)	must free every node	
write_json_array	O(n)	single traversal	
get_size	O(1)	counter maintained	

B. ItemBST – Binary Search Tree

1) Interface (tree.h):

```

1 struct ItemNode {
2     std::string id;
3     int row, col, value;
4     ItemNode* left;
5     ItemNode* right;
6 };
7
8 class ItemBST {
9     ItemNode* root;
10    int size;
11    ItemNode* insert_node(ItemNode* cur, ItemNode* node);
12    void clear_node(ItemNode* cur);
13    void write_inorder(ItemNode* cur, std::ostream& out,
14        bool& first) const;
15 public:
16    ItemBST();
17    ~ItemBST();
18    void insert(const std::string& id,
19        int row, int col, int value); // O(log n) avg
20    void inorder_json(std::ostream& out) const; // O(n)
21    void clear(); // O(n)
22    int get_size() const; // O(1)
23 };

```

Listing 12. ItemNode and ItemBST declarations.

2) *Ordering invariant*: Smaller values go left; larger values go right. Equal values use the item id string as a deterministic tie-breaker (lexicographically smaller id goes left). This guarantees consistent ordering even for maps with repeated values.

```

1 ItemNode* ItemBST::insert_node(ItemNode* cur,
2     ItemNode* node){
3     if (cur==nullptr) return node;
4     bool left = node->value < cur->value;
5     bool tie = node->value==cur->value &&
6         node->id < cur->id;

```

```

7     if (left || tie)
8         cur->left = insert_node(cur->left, node);
9     else
10        cur->right = insert_node(cur->right, node);
11    return cur;
12 }

```

Listing 13. Recursive insert with id tie-breaker.

```

1 void ItemBST::write_inorder(ItemNode* cur,
2     std::ostream& out, bool& first) const {
3     if (cur==nullptr) return;
4     write_inorder(cur->left, out, first);
5     if (!first) out << ",";
6     out << "{\"id\":\"" << cur->id
7     << "\",\"row\":\"" << cur->row
8     << "\",\"col\":\"" << cur->col
9     << "\",\"value\":\"" << cur->value << "}";
10    first = false;
11    write_inorder(cur->right, out, first);
12 }

```

Listing 14. Inorder traversal writing JSON array.

3) Complexity			summary:
Operation	Average	Worst case	
insert	O(log n)	O(n) (degenerate)	
inorder_json	O(n)	O(n)	
clear	O(n)	O(n)	
get_size	O(1)	O(1)	

In practice $n \leq 4$ (Hard difficulty), so worst-case behaviour is irrelevant for game performance; the BST is included to satisfy the course requirement and demonstrate the concept.

VII. PYTHON ALGORITHMS

A. Greedy Item Selector (greedy.py)

1) *Purpose and classification*: The function choose_greedy_item(state) selects exactly one uncollected item per turn to highlight in the UI. It is *greedy* because it commits to the locally best choice—based on value and immediate danger assessment—without simulating future moves or considering the overall route.

2) *Decision hierarchy*: Three priority levels are evaluated in order:

- 1) **High-value item near a guard, currently safe.** Items within Manhattan distance ≤ 2 of any guard trigger an alarm on pickup. Prioritising them while they are still outside the vision zone maximises expected score per alarm unit spent.
- 2) **Highest-value safe item.** If no near-guard safe item exists, the algorithm selects the uncollected item with the highest value that lies outside all vision zones.
- 3) **Warning fallback.** If every remaining item is inside a vision zone, the highest-value item is returned with a warning reason string.

```

1 NEAR_GUARD_DISTANCE = 2
2
3 def choose_greedy_item(state):
4     items = get_remaining_items(state)
5     guards = state["guards"]
6     zones = state.get("vision_zones", [])
7     if not items:
8         return {"item": None,
9             "reason": "There are no items to choose."}

```

```

10
11 # Priority 1: near guard AND safe
12 best_near = None
13 for item in items:
14     dangerous = is_in_vision_zone(item, zones)
15     near_guard = any(
16         manhattan_distance(item["row"], item["col"],
17                             g["row"], g["col"])
18         <= NEAR_GUARD_DISTANCE
19         for g in guards)
20     if near_guard and not dangerous:
21         if best_near is None or \
22            item["value"] > best_near["value"]:
23             best_near = item
24
25 if best_near:
26     return {"item": best_near,
27            "reason": "Highest-value item near a
28                    guard."}
29
30 # Priority 2: safest highest-value item
31 best_safe = None
32 for item in items:
33     if not is_in_vision_zone(item, zones):
34         if best_safe is None or \
35            item["value"] > best_safe["value"]:
36             best_safe = item
37
38 if best_safe:
39     return {"item": best_safe,
40            "reason":
41                "No safe item near guard; "
42                "highest-value safe item chosen."}
43
44 # Priority 3: fallback with warning
45 best = max(items, key=lambda i: i["value"])
46 return {"item": best,
47        "reason":
48            "Warning: all items are in a vision zone."}

```

Listing 15. choose_greedy_item() – full implementation.

3) *Complexity*: Let i = uncollected items, g = guards, v = vision-zone cells. Each item is checked against all guards ($O(g)$) and all vision cells ($O(v)$), giving $O(i(g + v))$ overall. With $i \leq 4$, $g \leq 3$, and $v \leq 18$ in Hard mode, the function runs in effectively constant time.

B. Backtracking Escape Router (backtracking.py)

1) Purpose and classification:

find_escape_route(state) performs a depth-first search from the player's current cell to the exit, respecting the full set of blocked cells. It follows the canonical *choose – explore – unchoose* backtracking pattern: a cell is added to the route before recursing and removed if that branch fails to reach the exit.

2) *Blocked-cell classification*: A cell is blocked if it is: walls, camera positions, guard positions, vision-zone cells, or any cell already in the current recursion path (to prevent cycles).

```

1 def find_escape_route(state):
2     rows = state["grid"]["rows"]
3     cols = state["grid"]["cols"]
4     start = state["player"]
5     goal = state["exit"]
6
7     blocked = []
8     for key in ["walls", "cameras", "guards"]:
9         add_positions(blocked, state[key])
10    add_positions(blocked, state.get("vision_zones", []))
11
12    route = []
13    visited = []
14    result = find_route_from(
15        start["row"], start["col"],
16        goal["row"], goal["col"],
17        rows, cols, blocked, visited, route)

```

```

18 return result if result is not None else []

```

Listing 16. find_escape_route() – setup and entry.

```

1 def find_route_from(row, col, gr, gc,
2                     rows, cols, blocked, visited, route):
3     if not is_valid_cell(row, col, rows, cols,
4                           blocked, visited):
5         return None
6     # Choose
7     visited.append((row, col))
8     route.append({"row": row, "col": col})
9
10    if row==gr and col==gc:
11        return route.copy() # goal reached
12
13    moves = get_ordered_moves(row, col, gr, gc,
14                              rows, cols, blocked, visited)
15    for dr, dc in moves:
16        result = find_route_from(
17            row+dr, col+dc, gr, gc,
18            rows, cols, blocked, visited, route)
19        if result is not None:
20            return result # propagate success
21
22    # Unchoose
23    visited.pop()
24    route.pop()
25    return None

```

Listing 17. Recursive backtracking core.

3) *Move ordering heuristic*: Candidate moves are sorted by Manhattan distance to the exit before recursing. This heuristic guides the search toward the goal first, dramatically reducing the number of nodes expanded in typical game states.

```

1 def get_ordered_moves(row, col, gr, gc,
2                       rows, cols, blocked, visited):
3     moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
4     valid = [(dr, dc) for dr, dc in moves
5              if is_valid_cell(row+dr, col+dc,
6                              rows, cols, blocked, visited)]
7     valid.sort(key=lambda m: manhattan_distance(
8         row+m[0], col+m[1], gr, gc))
9     return valid

```

Listing 18. Move ordering by Manhattan distance.

4) *Complexity*: Theoretical worst-case is $O(4^n)$ where n is the number of grid cells (up to $10 \times 10 = 100$ in Hard mode). In practice, walls and vision zones partition the grid significantly, and the Manhattan-distance heuristic makes the first path found nearly optimal in most game states. The route is recomputed after every move, running at the UI frame rate (60 fps cap).

VIII. PYGAME USER INTERFACE (MAIN.PY)

A. Window Layout

```

TILE_SIZE = 54 # px per grid cell
MARGIN = 28 # outer margin
PANEL_WIDTH = 340 # sidebar width
BOARD_SIZE = GRID_SIZE * TILE_SIZE
WINDOW_WIDTH = BOARD_SIZE + PANEL_WIDTH + MARGIN*3
WINDOW_HEIGHT = 720

```

Listing 19. Layout constants.

The board occupies the left portion of the window. The sidebar (PANEL_WIDTH=340 px) contains the title, status banner, metrics HUD, greedy recommendation, route length, and keyboard shortcuts.

B. Screen State Machine

The UI implements a lightweight state machine with five states:

TABLE IV
SCREEN STATES AND TRANSITIONS

State	Description / transitions
start	Difficulty selector. 1/2/3 change difficulty and reset; Enter starts; H goes to tutorial.
tutorial	Static help screen. B returns; Enter starts game; R resets.
playing	Active gameplay. Arrow/WASD move; H pauses to tutorial; R resets; M returns to menu.
escaped	Victory screen with final metrics. R or M available.
caught	Defeat screen with game-over reason. R or M available.

C. Rendering Pipeline

Each frame in the `playing` state renders in the following order to ensure correct visual layering:

- 1) **Floor tiles.** Checkerboard pattern in two alternating beige tones (`FLOOR_COLOR` / `FLOOR_ALT_COLOR`).
- 2) **Vision-zone overlay.** Semi-transparent orange surface (`SRCALPHA`) blitted over each vision cell.
- 3) **Backtracking route.** Cyan dots (`draw.circle`, radius 5) at each route cell centre.
- 4) **Walls.** Dark grey rectangles with a thin border.
- 5) **Items.** Purple diamond polygons (`draw.polygon`).
- 6) **Greedy outline.** Pulsing gold rectangle border around the selected item (pulse period ≈ 180 ms).
- 7) **Exit.** Green filled rectangle.
- 8) **Cameras.** Yellow triangle (`draw.polygon`).
- 9) **Guards.** Red filled circle (`draw.circle`, radius 16).
- 10) **Player.** Blue filled circle (`draw.circle`, radius 16).

D. Greedy Pulse Animation

```

1 pulse = (pygame.time.get_ticks() // 180) % 4
2 # pulse cycles 0->1->2->3->0 every 180 ms
3 # used as padding offset for the selected-item border
4 def draw_selected_item(screen, selected_item, pulse):
5     if selected_item is None: return
6     padding = 10 - pulse
7     rect = tile_rect(selected_item["row"],
8                       selected_item["col"], padding)
9     pygame.draw.rect(screen, SELECTED_ITEM_COLOR,
10                      rect, 4, border_radius=4)

```

Listing 20. Pulse calculation in main game loop.

E. Time Tracking and Tick Mechanism

Elapsed time is computed in Python and passed to C++ on every action. An additional "tick" action is sent whenever the displayed second advances, allowing the engine to enforce the time limit:

```

1 if screen_state=="playing" and status=="Playing":
2     elapsed = current_elapsed_seconds(start_ticks)
3     state["elapsed_seconds"] = elapsed
4     if elapsed != last_tick_second:
5         bridge.write_input("tick","",state,elapsed)
6         bridge.run_engine()
7         state = bridge.load_state()
8         greedy_result, escape_route = update_results(state)

```

```

9 last_tick_second =
10 state.get("elapsed_seconds",elapsed)
11 if state.get("game_over", False):
12     screen_state = "caught"

```

Listing 21. Tick mechanism in the game loop.

F. Sidebar HUD

The sidebar displays nine live metrics: Status, Difficulty, Score, Items collected / total, Moves / max, Time / limit, Alarm / max, Route length (backtracking), and Rank. The alarm value turns `ALARM_COLOR` (amber) when within one point of the limit and `DANGER_COLOR` (red) at the limit. The greedy-selected item and reason string appear below the metrics.

IX. TESTING AND VALIDATION

A. Test Plan

Table V lists all 18 manual test cases that were executed against the final build. Each test verifies one specific engine or algorithm behaviour.

TABLE V
COMPLETE VALIDATION TEST SUITE

ID	Component	Expected result
T01	Engine build	Compiles without warnings or errors.
T02	Pygame launch	Window opens at correct size.
T03	Valid move	Player advances one cell; history grows.
T04	Wall collision	Player stays; alarm increments by 1.
T05	Out-of-bounds	Player stays; no alarm change.
T06	Item pickup	Score increases by item value; item disappears.
T07	Near-guard pickup	Alarm increments on collection.
T08	Vision zone	Immediate game-over; reason = <i>Caught in vision zone</i> .
T09	Alarm limit	Game-over when alarm $\geq$ max_alarm.
T10	Move limit	Game-over when history length $\geq$ max_movements.
T11	Time limit	Game-over on tick when elapsed $\geq$ time_limit.
T12	Escape	status = <i>Escaped</i> ; victory screen shown.
T13	Rank A	Achieved with score $\geq 70\%$, half moves, half time, alarm ≤ 1 .
T14	Restart (R)	All metrics reset; map re-loaded.
T15	Main menu (M)	Returns to start screen without error.
T16	Greedy output	Returns correct item id and reason string.
T17	Backtracking	Returns valid non-colliding route or empty list.
T18	BST inorder	items_sorted_by_value ascending by value.

B. Hard-Mode State Verification

The `state.json` produced by a Hard-mode reset was inspected to confirm the BST inorder output. With items: coin (25), vase (60), mask (80), painting (100), the expected ascending order is:

```

[
  {"id":"coin",    "row":8,"col":3,"value":25},
  {"id":"vase",    "row":6,"col":1,"value":60},
  {"id":"mask",    "row":1,"col":8,"value":80},

```

```
{ "id": "painting", "row": 2, "col": 6, "value": 100 }
}
```

Listing 22. items_sorted_by_value from Hard-mode state.json.

This matches the inorder traversal of the BST built in `write_state_file()`, confirming the tree ordering invariant.

X. DISCUSSION

A. Algorithms as Game Mechanics

The greedy algorithm gives the player an immediate, visible recommendation each turn, making the concept of “local optimality without global guarantee” tangible: the highlighted item is not always the one that maximises the final score because it ignores route cost and alarm accumulation. The backtracking route changes in real time as the player moves and as vision zones shift the set of blocked cells, demonstrating the choose-explore-unchoose pattern dynamically.

The linked list’s tail-pointer optimisation ($O(1)$ append) is a deliberate design decision that can be demonstrated live: every move appends exactly one node. The BST’s inorder output produces the sidebar “items by value” list automatically, making the connection between the tree structure and its sorted output concrete.

B. Limitations

- **Hardcoded maps.** All three maps are compiled into the binary. Adding a map requires recompilation, limiting replay value.
- **Manual JSON parser.** The `get_string_value / get_int_value` helpers are fragile against unexpected whitespace or nested keys. A malformed file can produce undefined behaviour.
- **Unbalanced BST.** Inserting items in value order (e.g. $25 \rightarrow 60 \rightarrow 80 \rightarrow 100$) produces a right-skewed tree with $O(n)$ insert in the worst case. An AVL or red-black tree would guarantee $O(\log n)$.
- **Backtracking does not optimise score.** The route found is the first feasible path to the exit, not the one that passes through the most valuable items.
- **Static guards.** Guards do not patrol; their positions are fixed for the entire run, which reduces strategic depth.

C. Future Work

- Procedural map generation with a configurable seed.
- Replace the manual JSON parser with `nlohmann/json`.
- Implement an AVL tree to eliminate worst-case $O(n)$ insert.
- Add guard patrol paths with periodic vision recalculation.
- Extend backtracking to maximise collected item value along the route (branch-and-bound or A* with a value heuristic).

XI. CONCLUSION

Museum Heist successfully integrates C++17, Python 3, Pygame, and JSON inter-process communication into a playable stealth game where each course concept is a live, observable mechanic. The C++ engine provides a memory-managed, compiled backend with a custom linked list and BST; the Python layer provides a high-productivity environment for rendering and algorithm prototyping. The file-based IPC makes the data flow transparent and debuggable. Three difficulty levels, five screen states, a greedy recommendation, a real-time backtracking route, and a full rank system combine to produce a project that is both technically sound and engaging to play. All 18 planned test cases passed on the final build.

REFERENCES

- [1] C. A. Sierra, *Computer Sciences I: Course Project Description*, Universidad Distrital Francisco Jose de Caldas, 2026-I.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [3] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. Springer, 2008.
- [4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [5] Pygame Community, *Pygame Documentation*, 2024. [Online]. Available: <https://www.pygame.org/docs/>
- [6] ISO/IEC, *Programming Languages – C++*, ISO/IEC 14882:2017 (C++17), 2017.